Day 10 — 전체 흐름 잡기 + RAG를 백엔드에 연결

> **"부품만 만드는 것 같고 전체가 안 보인다"** → 이 문서 하나로 큰 그림을 잡는다. 코드 치기 전에 **0번 섹션(전체 흐름)** 부터 읽자. 이거 이해하면 지금까지 만든 게 다 한 줄로 꿰인다.

🗺️ 0. 이 프로젝트가 통째로 어떻게 돌아가나

한 문장

> **강사가 화면에서 아이를 고르고 오늘 수업을 입력하면 → 서버가 그 아이 정보·최근 기록·RAG 지도자료를 모아 LLM한테 주고 → LLM이 맞춤 피드백을 써서 → 저장하고 화면에 보여준다.**

이 한 문장 안에 우리가 만든 부품이 **전부** 들어있다. 하나씩 풀어보자.

등장인물 (부품) 5개

부품	정체	비유	우리 파일
Streamlit	강사가 보는 **화면**	식당 홀 (손님이 앉는 곳)	아직 안 만듦 (Day 11~12)
FastAPI	요청을 받아 처리하는 **서버**	웨이터 (주문 받고 주방에 전달)	`main.py` + `routers/`
PostgreSQL(DB)	정보를 쌓아두는 **창고**	냉장고·장부	`models.py` + `database.py`
LLM(제미나이)	피드백 글을 **쓰는 AI**	요리사	`llm_service.py`
RAG	우리만의 지도자료를 **찾아주는** 검색기	요리사 옆 레시피북	`rag/` (kids.py·search.py)

👉 **핵심 감각:** 하나가 다 하는 게 아니라, **부품끼리 주고받으며** 일이 굴러간다. 웨이터(FastAPI)가 중심에서 창고(DB)·요리사(LLM)·레시피북(RAG)을 부린다.

실제로 한 번 요청이 흐르는 순서 (피드백 생성)

```
① 강사가 화면(Streamlit)에서 "ㅇㅇ이 오늘 수업" 입력 + [피드백 생성] 클릭
        │  (아직 화면 없음 → 지금은 Swagger로 대신 테스트)
        ▼
② FastAPI 서버가 요청 받음   →  routers/feedbacks_router.py 의 /generate
        │
        ├─▶ ③ DB 창고에서 꺼내기
        │       - 이 수업기록 1개 (record)
        │       - 그게 누구 아이인지 (child)
        │       - 그 아이 최근 기록 5개 (records)
        │
        ├─▶ ④ 【Day 10 신규】 RAG로 지도자료 찾기
        │       - 아이 상황을 질문으로 만들어 search 로직에 던짐
```

```
        │       - vectors.json에서 뜻으로 가장 가까운 조각(가이드) 꺼냄
        │
        ├─▶ ⑤ 프롬프트 조립   →  prompt_service.py 의 build_prompt()
        │       아이정보 + 최근기록 + 【④에서 찾은 지도자료】 를 글 한 덩어리로
        │
        ├─▶ ⑥ LLM 호출        →  llm_service.py 의 generate_feedback()
        │       조립한 프롬프트를 제미나이한테 → JSON 답 받음
        │       { next_goal, teacher_feedback, parent_feedback }
        │
        └─▶ ⑦ DB 창고에 저장   →  Feedback(...) → db.add → commit
        ▼
⑧ 결과를 화면(Streamlit)에 카드로 보여줌  (Day 12)
```

지금 ④만 비어있다. ①~③, ⑤~⑦은 이미 만들어놨고(1주차), RAG도 따로 완성해뒀다(Day 9). Day 10이 하는 일은 **④를 만들어서 기존 흐름에 끼워넣는 것.** 그래서 "결승선"이라 부르는 거야.

파일이 이렇게 나뉜 이유 (왜 한 파일에 안 넣나)

각 부품이 **자기 일 하나만** 하도록 파일을 쪼갰다. 이걸 "역할 분리"라고 해.

```
main.py                ← 서버 켜기 + 라우터(주문 창구)들 등록
database.py            ← 창고(DB) 연결 설정
models.py              ← 창고에 뭘 어떤 모양으로 쌓을지 (테이블 설계도)
schemas.py             ← 주고받는 데이터 모양 검사 (입력/출력 양식)
routers/
  children_router.py       ← 아이 등록·조회 창구
  lesson_records_router.py ← 수업기록 창구
  feedbacks_router.py      ← 피드백 생성·조회 창구  ★여기가 ④ 낄 자리
prompt_service.py      ← 주문서(프롬프트) 작성 담당
llm_service.py         ← 요리사(LLM)한테 주문 넣고 받아오기
rag/
  kids.py              ← RAG 준비: 자료 자르고→숫자로→vectors.json 저장 (한 번만)
  search.py            ← RAG 검색: 질문 → 가장 가까운 조각 찾기
```

💡 **왜 쪼개나:** 나중에 한 군데 고칠 때 딴 데 안 건드려도 되니까. 요리사(LLM)를 바꿔도 웨이터(FastAPI)는 그대로. 이게 "부품으로 나눠 만든" 진짜 이유야.

지금 네가 헷갈리는 이유 (정상이야)

너는 지금 **부품을 하나씩** 만들어왔어:

- 1주차: 웹 서버 쪽 부품 (창구·창고·주문서·요리사 호출)
- 2주차 Day 8~9: RAG 부품 (자료 검색기) — 근데 이건 **웹 서버랑 따로**, `rag/` 폴더에서 **혼자 도는 실험 스크립트**로 만들었어.

그래서 "두 세계"가 아직 안 만난 상태라 전체가 안 보이는 거야. **Day 10이 그 두 세계를 연결하는 다리**야. 이거 하고 나면 "아 이래서 그 부품들을 만들었구나" 하고 딱 붙어.

🎯 1. 그래서 오늘(Day 10) 하는 일

큰 목표

`rag/search.py`에 흩어진 **검색 로직**을, 피드백 만드는 흐름에 **끼워넣는다.**

오늘의 첫 세션 (결승선 = 개념 증명)

아직 웹(FastAPI)에 바로 안 붙이고, **먼저 `search.py` 안에서** "찾기 → LLM에 먹여 답 만들기"까지 되는지 확인한다. (작게 닫고 → 다음에 웹에 옮김)

```
[이미 됨]  질문 → 가장 가까운 조각 찾기 (search.py)
    +
[오늘]     찾은 조각을 프롬프트에 끼워 → generate_feedback() 호출 → 최종 답
```

- `found = data[best]["text"]` — 찾은 조각을 변수에 담기
- `generate_feedback` 가져와서 재사용 (Day 4에 만든 것, `llm_service.py`에 있음)
- 프롬프트 f-string: "아래 [가이드] 참고해서 이 [상황]에 피드백 써줘" + found + question
- 호출 → print

그다음 (오늘 다 못 하면 다음 세션)

`feedbacks_router.py`의 ④ **자리**에 검색을 넣어서, 진짜 웹 API가 RAG까지 써서 답하게 만들기.

---

2. 🏁 오늘 한 것 — RAG 결승선 통과 (search.py 안에서 검색→생성)

완성한 흐름 (search.py 맨 밑)

```
found = data[best]["text"]          # ④ 검색 결과 (Day 9에 만든 3단계가 뽑아준 조각)

prompt = f"""너는 아이 줄넘기 수업 강사를 돕는 AI야.
아래 [지도 가이드]를 근거로, [상황]에 맞는 피드백을 짧게 써줘.

[지도 가이드]
{found}          ← RAG의 심장: 찾은 자료를 프롬프트에 끼워넣음

[상황]
{question}
"""                                 # ⑤ 프롬프트 조립

answer = generate_feedback(prompt)  # ⑥ LLM 호출 (Day 4에 만든 함수 재사용)
print(answer)
```

→ 결과: 위축형 상황 질문 → 우리 지도자료를 **근거로** 한 피드백이 나옴 ✅

오늘 이해한 핵심 개념

- **RAG 검색은 이미 만들어놨다** = Day 9 ④의 3단계(질문 숫자화 → 8개와 cos_sim → max/index로 1등). 새로 만드는 게 아니라 **함수로 포장해서 웹에 옮기면 됨.**
- `{found}`**가 RAG의 심장** — LLM은 우리 자료를 모르지만, 프롬프트에 넣어주는 순간 그걸 보고 답한다.
- **순서는 바뀔 수 없다: 검색(④) → 프롬프트(⑤) → LLM(⑥).** 뒷 단계가 앞 결과를 재료로 쓰니까.
- **고정값 vs 바뀌는 값:** `question = "소심한 애..."`는 지금 **실험용 고정값.** 진짜 코드에선 매개변수로 받거나 들어온 수업 기록으로 조립해서 매번 바뀐다.
- **큰 그림:** search.py = 피드백 8단계 중 ④(검색) 담당 부품. 아직 웹(feedbacks_router)엔 안 붙음.

🐞 오늘 헷갈린 것

- `found = ...`처럼 `=` **왼쪽은 "만들기"** 라 앞에 없어도 에러 안 남. 없는 걸 **꺼내 쓸 때만** NameError.
- `{found}`, `{question}`은 다른 파일에서 오는 게 아니라 **같은 파일(search.py) 위쪽에서 태어난 변수.**

🧩 추가로 한 것 — 검색을 `rag_search()` 함수로 포장

```
def rag_search(question):
    q_vector = embed_feedback(question)
    scores = []
    for d in data:
        scores.append(cos_sim(q_vector, d["vector"]))
    best = scores.index(max(scores))
    found = data[best]["text"]
    return found

question = "소심한 애가 처음으로 발표했는데 뭐라고 해줘야 하죠?"
found = rag_search(question)      # ★함수 호출★ → 찾은 조각 받기
```

- **왜 포장하나:** 겉 결과는 똑같지만, `rag_search(질문)` 한 줄로 부를 수 있는 **재사용 부품**이 됨 → 웹서버 ④ 자리에 옮길 준비 완료.
- 🐞 포장할 때 막힌 것 3개 (전부 위치 문제): ① `def` 앞 공백 → IndentationError ② `best`·`found`가 함수 밖으로 나감(들여쓰기 빠짐) → `scores` 안 보임 ③ `return found` 빠뜨림 → 함수가 값을 안 돌려줌. + 함수는 "만들기"라 아래서 **호출**(`found = rag_search(question)`)해야 실제로 일함.
- 🐞 "에러 난다" → 코드 문제 아니고 **실행 방법** 문제였음. IDE ▶ 버튼(기본 python)이 아니라 **venv 파이썬**으로 돌려야 함: `Kids_Feedback\.venv\Scripts\python.exe ...`

⏭️ 다음 세션 (오늘 손대지 말 것)

- `rag_search`를 `feedbacks_router.py` ④ 자리에 끼우기
- `build_prompt`에 `found` 받을 자리 하나 추가